

PISK Documentation

Pillo Input Simulation Kit

Introduction

PISK (Pillo Input Simulation Kit) is a Unity tool designed to simulate pressure input from a [Pillo](#). Testing with a real Pillo can be challenging due to its physical nature, availability, and setup time. PISK bridges that gap, allowing developers to simulate pressure input directly within the Unity editor for faster prototyping, iteration, and debugging.

Why use PISK?

The Pillo Input Simulation Kit (PISK) boosts productivity and simplifies testing by offering a fast, intuitive way to simulate pressure input, no hardware required. Its lightweight design and easy-to-use interface make it compatible with nearly all Unity versions and project setups.

You can integrate PISK into your project in seconds, making it an ideal tool for rapid prototyping, development, and debugging of Pillo-based apps.

Getting Started

Setting up the panel

PISK has been designed and developed in a way where minimal configuration is required.

After importing the package, go to **Plugins/PISK/Prefabs** and drag the **PISK Controller prefab** into your Scene Hierarchy. The simulation panel will be ready to use with no further set up required.

Simulating pressure

Press or hold the **spacebar** to increase the pressure. Releasing the spacebar will cause the pressure to gradually return to its minimum value.

Configuring parameters

PISK provides adjustable parameters directly in the simulation panel. You can tweak values like Press, Release, Threshold, and more to control how the pressure behaves. Changes take effect in real-time during Play mode for quick iteration. For detailed descriptions of each setting, see the **Understanding the PISK UI** section.

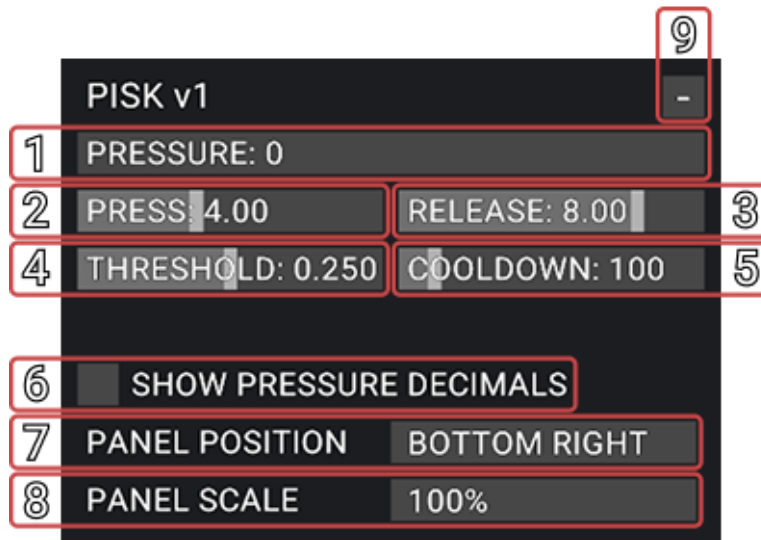
Accessing the global pressure variable

To use the simulated pressure in your scripts, access it through the instance of the controller:

```
float pressure = PISKController.Instance.Pressure;
```

This gives you the current pressure value, which you can use just like input from a real Pillo device, perfect for testing gameplay. An example of how to use the pressure value in context is included in the **Plugins/PISK/Scene/ExampleScene**.

Understanding the PISK UI



1. Pressure

Represents the simulated pressure value.

The rate of change is controlled by the Press and Release values. Min and max pressure bounds are configurable in the Unity Editor (values will not update properly if configured at runtime).

2. Press

Controls how quickly pressure increases while input is held. The higher the value, the faster the pressure increases. Setting this value to 0 will prevent any pressure from building.

3. Release

Controls how quickly pressure decreases after input is released. The higher the value, the faster the pressure returns to its minimum. Setting this to 0 will cause pressure to remain constant after release.

4. Threshold

Sets the *minimum value* for Press and Release. This prevents very low values from causing slow or unnoticeable changes in pressure over time. This value can be set to 0 to completely remove the threshold.

5. Cooldown

Adds a delay (in milliseconds) after input is released, during which the pressure remains at its current value before starting to decrease. The higher the value, the longer the delay. Setting this value to 0 means there will be no cooldown.

6. Show Pressure Decimals

Toggles whether the pressure value is displayed with two decimal places. Useful for more precise fine-tuning and debugging.

7. Panel Position

Allows you to choose the corner of the screen where the PISK panel is displayed. This makes it easier to position the panel to avoid overlapping with other UI elements in your scene.

8. Panel Scale

Sets the size of the panel interface as a percentage. Higher values increase the panel's size, which can help with visibility or presentations. For example, setting it to 200% will double the panel size.

9. Panel Toggle Button

Lets you minimize or expand the PISK panel. Useful for keeping the interface out of the way when not adjusting settings. Minimizing the panel does not affect the pressure simulation.